

Corelia — 技術的こだわり

2.5D × マリオギャラクシー風の重力アクション

当たり判定 / 重力 / カメラ を中心に

① 技術的こだわり（全体像）

コンセプト：「Y固定をやめ、すべてをベクトルと形状で考える」

当たり判定	球 vs 形状の押し出し。地形のローカル空間で解く → 回転(OBB)も同一コードで対応
重力	惑星ごとの重力 + 手動重力。upDir が全処理（移動 / ジャンプ / カメラ）の基準
カメラ	追従Lerp + 重力対応回転 + ルーム遷移 + ジュース（揺れ / 傾き / ズーム）

見せ場

- 手作りプロシージャルメッシュを DrawVertices で描画（アウトライン = 裏面CullFront）
- グレースケール × 乗算色で レインボー着色
- カーソルを逆投影してワールド化 → 岩を吸い寄せ & 発射 → 壁で割れる

② 縁の下の作り込み

- 仮想解像度：バックバッファ固定でUI非依存／ImGuiは `FramebufferScale` で補正
- FPS非依存：アニメ・エフェクト・トレイルを `dt×60` ／実時間で統一
- BGMこもり同期：通常版＋こもり版を同時再生しクロスフェード（ポーズで頭出しに戻らない）
- 起動ロード画面：ブロッキング読込の合間に手動Present＋進捗バー
- データ駆動エディタ：ImGuiで配置 → ステージ別CSV保存（取得=Collect / 削除=Expire を分離）

③ Sphere惑星の重力

考え方：「惑星の中心へ引っばられる」だけ

計算の流れ（XY平面・2.5D）

- 中心→プレイヤーのベクトル $\mathbf{v} = \text{player} - \text{center}$
- 正規化して上方向 $\text{upDir} = \mathbf{v} / |\mathbf{v}|$ （表面の真上）
- 重力はその逆 $\text{gravDir} = -\text{upDir}$ （中心向き）
- 強さは距離の2乗に反比例（万有引力風）
- 重力圏の端で smoothstep で0へ（ふわっと離脱）
- 複数惑星はベクトル合成、最強の惑星のupDirを採用

- プレイヤー
↑ upDir（外向き）
|
▼ gravDir（中心向き = 重力）
(●) 惑星の中心

Box惑星は「最も近い面の外向き法線」を upDir にするだけ。

どちらも『upDirを決めて $\text{gravDir} = -\text{upDir}$ 』という同じ設計に乗る。

④ Kdへの独自拡張（骨組みからの差分）

追加7・変更47ファイル／とくに「アウトライン」と「重力対応の草・球法線」が独自の描画拡張

描画シェーダ（StandardShader）

- **アウトライン描画**：専用VS/PS新規 +
BeginOutline/SetOutlineWidth（裏面押し出し）
- **草ブレンド**：GravityUpDir 基準で地面に草を生成
（SetGrassBlend 等）
- **トライプラナーUV／SphereNormal**（球惑星の法線を中心基準で
計算）
- **DrawVertices 拡張**：トポロジー／深度／カリング指定

スプライト（SpriteShader）

- **角丸UI**：DrawRoundedBox / RoundedTex / RoundedBubble
- 日本語フォント描画版 DrawFont

UI / デバッグ（KdDebugGUI）

- **常時ImGuiコールバック**／日本語化・FramebufferScale補正
- **Gameビューポート**（エディタ内にゲーム描画）

土台・ユーティリティ

- **KdFPSController**：static GetDt() でdtをグローバル化（FPS非依存
の土台）
- **KdEasing**：Sine/Expo/Cubic/Bounce 追加
- **KdCollider**：SetNodeFilter（ノード単位の判定）
- **KdThreadPool** 新規／KdEffekseer・KdWindow(ホイール) 改変

④-A アウトライン描画（2パス法）

本体の外側に黒いシルエットを1枚先に描き、その内側を本体で塗りつぶして“縁だけ”残す。

処理の流れ

- **パス1（縁）**：頂点を**法線方向に Width だけ押し出し**、CullFront で**裏面だけ**を単色（黒）で描画 + 深度書き込み
- **パス2（本体）**：通常のLit描画が内側を上書き → 押し出したぶんの輪郭が残る

技術詳細

- 専用シェーダ `VS_Outline`（頂点押し出し）／`PS_Outline`（単色）を新規追加
- 定数バッファ `cbOutline { Width, DepthBias }`、`SetOutlineWidth()`で幅指定（既定 `Width=0.04` = ワールド単位なので遠近で太さが一定）
- 切替は `BeginOutline()/EndOutline()`（ラスタライザを `CullFront` に）
- 岩は `DrawVertices()` を `Scale×OutlineScale + CullFront` で呼んでも同じ効果

④-B 草ブレンド（重力対応・PS内）

面の法線と重力の上方向の内積で「上向き＝草／横向き＝岩」をピクセル単位で混ぜる。

計算式（ピクセルシェーダ）

- $d = \text{dot}(N, \text{GravityUpDir})$ （面が上をどれだけ向くか, $-1 \sim 1$ ）
- $t = \text{smoothstep} / \text{pow}(d, \text{Sharpness})$ で $0 \sim 1$ 化
- $\text{albedo} = \text{lerp}(\text{岩テクスチャ}, \text{草テクスチャ}, t)$
- 境界帯 `GrassEdgeWidth` に専用エッジテクスチャを重ねる

技術詳細

- 草は**トライプラナー**でサンプリング（UVなしメッシュでも貼れる）
- `GravityUpDir` を毎フレーム定数バッファ `cb0_Obj` へ転送 → **惑星が回転しても“上”が追従**
- API : `SetGrassBlend(enable, upDir, sharpness, scale) / SetGrassTexture / SetGrassNormalTexture`（法線マップ） / `SetGrassEdgeTexture`
- フラグ : `UseGrass / UseGrassNormal / UseGrassEdge`

④-C トライプラーUV / 球の法線

トライプラー投影

- ワールド座標をXY・YZ・ZX の3平面へ投影して3回サンプリング
- 重み $w = \text{pow}(\text{abs}(N), k)$ を正規化してブレンド
- → UV展開不要・継ぎ目（シーム）が出ない。
TriplanarScale=0.1

球の法線 (SphereNormal)

- PSで法線を $N = \text{normalize}(\text{worldPos} - \text{center})$ に置換
- → ローポリ球でもカクつかず滑らかな陰影

🔧 技術詳細

- メッシュ法線を使わず面位置から法線を再構成するのがポイント（頂点数を増やさず丸く見せる）
- API : `SetTriplanarUV(enable, scale) / SetSphereNormal(true)`
- 重力の球惑星の地面描画で `SphereNormal` + 草ブレンド を併用
- 箱惑星は通常法線、球惑星のみ中心基準に切替

④-D DrawVertices の拡張

モデルを介さず、コードで作った頂点配列を直接GPUへ送って描く関数。引数で描画状態を選べるよう拡張。

シグネチャ

```
DrawVertices(  
    vertices,           // 頂点配列  
    mWorld,             // ワールド行列  
    colRate,            // 乗算色(着色)  
    depthState,         // 深度の扱い  
    topology,          // 線/三角形  
    raster);           // カリング
```

内部処理

- `cb1_Mesh.mW = mWorld` と `colRate` を定数バッファへ転送
- `ChangeRasterizerState` / `ChangeDepthStencilState` でステート切替 (描画後 Undo)
- 動的頂点バッファに **DISCARD 書き込み** → `context->Draw()` (インデックス無し)

選べる値

- `topology` : `TRIANGLELIST`(面) / `LINELIST`(線)
- `raster` : `CullNone`/`Front`/`Back` `depth` : `ZEnable`/`ZWriteDisable`
- 岩は**3回呼ぶ** (アウトライン + 面 + ワイヤー)

④-E 角丸UI / 日本語フォント

角丸の作り方（頂点生成）

- 角丸矩形 = 中央 + 上下左右の矩形 + 四隅の扇形で塗る
- 四隅は cornerSegs 分割した三角形ファンで円弧を近似
- radius は $\min(\text{extentX}, \text{extentY})$ でクランプ
- DrawRoundedBubble(To) : 角丸 + 三角の尻尾を対象座標へ向ける

🔧 日本語フォント DrawFont

- GDI で文字をテクスチャ化 (CP932) し、グリフごとにスプライト描画
- 中央寄せは 各グリフ幅の合計 を測って $-w/2$ ずらす
- 2系統 : KdFont=GDI/CP932 (DrawFont) と ImGui=UTF-8 を使い分け

用途 : 設定ウィンドウ、コリアのセリフ、吹き出しヒント。

④-F 開発用メニュー（ImGui）まわり

常時ImGuiコールバック

- GuiProcess() の NewFrame() 後に if(callback) callback() を
毎フレーム呼ぶ
- F3エディタの ON/OFF に関係なく描ける（SE EditorのF5トグル等）

Game ビューポート

- ポストプロセスのSceneRT を ImGui::Image で表示
- ホバー判定・UV取得でウィンドウ内のクリック位置を逆算

🔧 FramebufferScale 補正

- バックバッファを設計解像度1280×720に固定（＝仮想解像度）
- 実ウィンドウが別サイズだとImGuiだけ座標がズれる
- → `io.DisplayFramebufferScale =`
`(backbufferW/DisplaySize.x, h/y)` で補正
- マルチビューポート（ウィンドウ外ドラッグ）は維持したまま解決

④-G 土台のしくみ

GetDt (FPS非依存の核)

- KdFPSController が毎フレーム static s_deltaTime を更新
- 全クラスから KdFPSController::GetDt() で参照
- 各処理は 速度 $\times = dt \times 60$ で**60fps基準に正規化** → FPSを変えても同速

Easing

- 0→1 を非線形変換。例 InOutCubic = $t < .5 ? 4t^3 : 1 - (-2t+2)^3 / 2$
- カメラ寄り・UIのポップに使用 (等速のカクつきを消す)

🔧 NodeFilter / ThreadPool

- **NodeFilter** : KdCollider の判定対象を**特定メッシュノード**だけに限定 (SetNodeFilter(indices)) → 体の一部位のみ当たり
- **ThreadPool** : ワーカースレッド群 + タスクキュー。Init() で起動し**重い処理を並列化**

dt×60 の徹底で、アニメ・Effekseer・トレイルすべてがFPS非依存に。

⑤ カリング処理と負荷軽減

カリング処理

- 基本は背面カリング（CullBack）で見えない裏面を描かない
- アウトラインは逆に CullFront：裏面だけ残して縁取りに（カリングを演出へ転用）
- 条件カリング：取得済み／期限切れアイテム・メニュー中のHUDは描画スキップ

処理負荷軽減 — 「作らない・描かない・使い回す」

- 共有メッシュを1回だけバイク：岩・重力コアは全インスタンスで頂点を使い回し
- ローポリ設計／動的頂点バッファのプールでバッファ再確保を削減
- 音声プリロードで実行時のカクつきを排除
- 死亡オブジェクトの自動掃除（remove_if）でリスト肥大を防止
- 重力は重力圏外を早期スキップ + 距離²フォールオフ ／ 投擲物は寿命で自動消滅

⑥ マルチスレッド（オブジェクトの並列更新）

たくさんのオブジェクトの更新を、CPUの複数コアで“同時に”走らせています。

しくみ

- 各オブジェクトの重い処理を `ParallelUpdate()` に分けて書く
- それを **KdThreadPool**（あらかじめ起動した作業スレッド群）にタスクとして投げる
- `std::future` で**全タスクの完了を待ち合わせ**てから次の処理へ進む
- 順序が大事／軽い処理は、通常の `Update()` で1個ずつ（逐次）

コード（BaseScene）

```
for (auto& obj : m_objList)
    jobs.push_back(
        KdThreadPool::Enqueue(
            []{ obj->ParallelUpdate(); }));
for (auto& f : jobs) f.get(); // 完了待ち
```

- スレッドを毎回作らず**使い回す**（生成コスト削減）
- 並列に置くのは**他と干渉しない処理だけ**（データ競合を避ける）

🔪 発表ではこう言う

オブジェクトの更新は**1個ずつ順番**にではなく、**スレッドプールに配ってCPUの複数コアで同時に**処理しています。全部終わるのを待ってから次に進むので、**数が増えてもコアの数だけ手分け**できて軽いままです。

⑦ オブジェクトが大量でも軽い理由

結論：オブジェクトプールは使っていません。「並列・共有・掃除」の合わせ技で軽くしています。

- ① 並列更新：重い更新をスレッドプールで複数コアに分散（前ページ）
- ② メッシュを1回だけベイクして全インスタンスで共有：岩や重力コアが何個でも頂点データは1セット
- ③ ローポリ設計 + 頂点バッファの再利用：毎フレームのメモリ確保を避ける
- ④ 更新を丸ごと停止：ポーズ・会話・ヒットストップ中はUpdate自体をスキップ
- ⑤ 死んだものは掃除 & 描かない：取得済み/期限切れは `remove_if` で除去・描画スキップ、投擲岩は寿命で消滅
- ⑥ パーティクルはEffekseer(GPU)任せ：CPU側のバーストも寿命・数で上限

🔪 発表ではこう言う

よく「プールで軽いの？」と聞かれますが、**使っていません**。軽い理由は、**更新をマルチスレッドで分担し、メッシュは1回だけ作って全部で使い回し、死んだ物は掃除して描かない**から。もし毎フレーム何百個も生成・消滅するようになれば、**次の一手としてオブジェクトプール**を入れます。